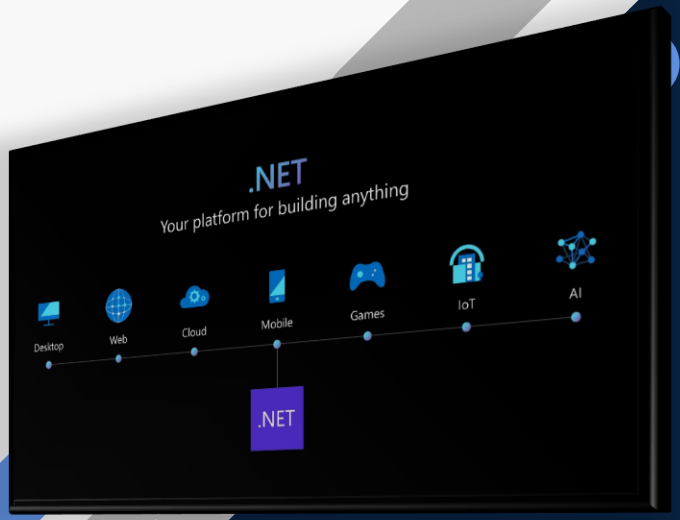
.NET

NETWORK ENABLED TECHNOLOGIES



Created by:

TECH TITANS

WHAT IS .NET?

- **Microsoft .NET** is a software development platform that helps developers build various applications and connect information, people, systems, and devices.
- .NET provides:
  - ✓ A runtime to execute programs
  - ✓ A huge library of ready-made functionalities
  - ✓ Support for multiple programming languages
  - ✓ Tools to build web, desktop, mobile, cloud, iot, and gaming applications
- A major architect tied to the success of .NET's key components is **Anders Hejlsberg**.

HISTORY OF .NET…

- **Pre-.NET technologies:** Before .NET, Windows applications relied on technologies such as the Win32 API, VB6 (Visual Basic 6), and C++.
- **Drawbacks:** These older technologies were characterized as hard to maintain, inconsistent, missing essential features like garbage collection, and complex for beginners.

2000 – .Net announced	Microsoft introduced the .Net framework, c#, and clr to simplify development and compete with java
2002 – .Net framework 1.0	Released with CLR 1.0, base class library, ASP.NET, windows forms, and support for C# & VB.Net.
2016 – .Net core	Cross-platform, open-source, fast, and lightweight—runs on windows, linux, and macos.
2020 – .Net 5	Unified .Net framework, .Net core, and xamarin into one modern platform.
Today – .Net 10	A unified, cross-platform, high-performance framework for web, mobile, desktop, cloud, and AI apps.

MICROSOFT C#

- C# (pronounced c-sharp) is a modern, object-oriented programming language created by Microsoft for building powerful, safe, and high-performance applications on the .NET platform.
- It is designed to be easy to learn, clean, and structured while offering advanced features for real-world software development.
- C# provides:

  A runtime to run your code

  A library of ready-made functions

  Tools to make development fast and easy

WHY .NET?

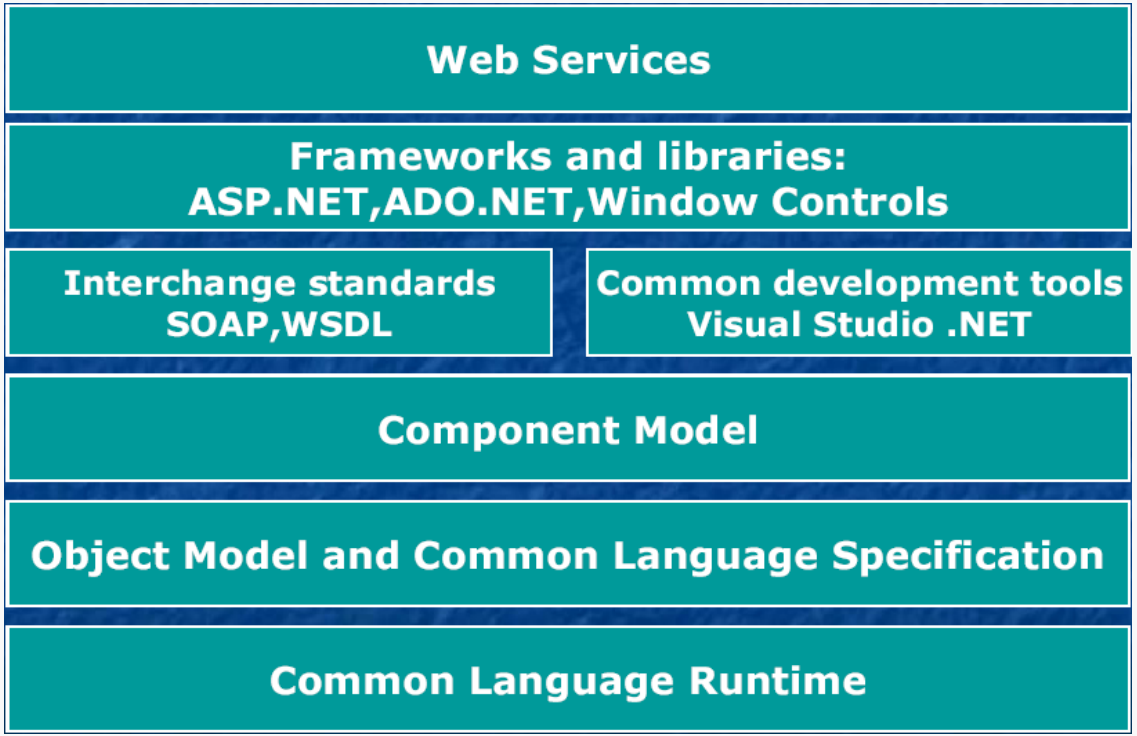
PROBLEM

- ✓ Hard to build apps in older Windows technologies
- ✓ No built-in memory management
- ✓ No consistent language model
- ✓ Windows-only limitations
- ✓ Poor security
- ✓ Need for fast, scalable apps

HOW .NET SOLVED IT

- ✓ Provided unified library and runtime
- ✓ Introduced Garbage Collection
- ✓ Introduced CLS, CTS, CLR
- ✓ Introduced .NET Core → Cross platform
- ✓ Introduced Code Access Security
- ✓ Modern JIT compiler, optimized runtime

.NET ARCHITECTURE



.NET ARCHITECTURE COMPONENTS

01

CLR (Common Language Runtime)

The heart of .NET

- ✓ Executes code
- ✓ Manages memory
- ✓ Handles security
- ✓ Performs garbage collection

02

BCL (Base Class Library)

A collection of ready-to-use features

- ✓ A collection of ready-to-use features:
- ✓ Networking & Encryption
- ✓ Collections (List, Dictionary)
- ✓ Math & UI controls

03

Languages

All languages compile into MSIL/IL and run on CLR.

- ✓ C# & F#
- ✓ VB.NET
- ✓ Powershell
- ✓ IronPython & IronRuby

WHY ONLY THESE LANGUAGES SUPPORT .NET?

They follow CLS (Common Language Specification)

Rules for all .NET languages.

They compile to IL (Intermediate Language)

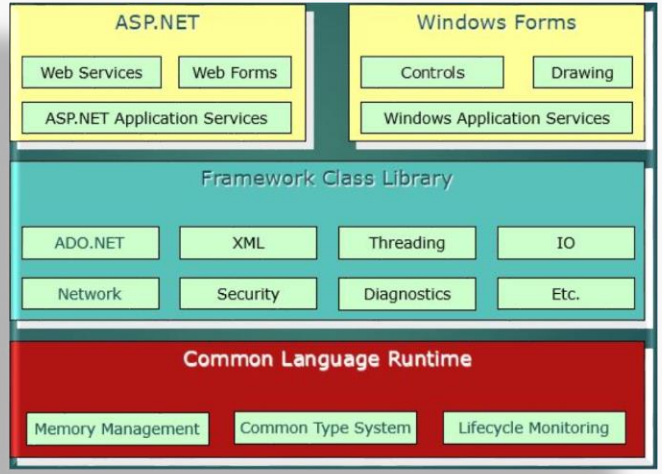
Then CLR executes it.

They follow CTS (Common Type System)

So a class written in C# can be used in VB.NET and vice versa.

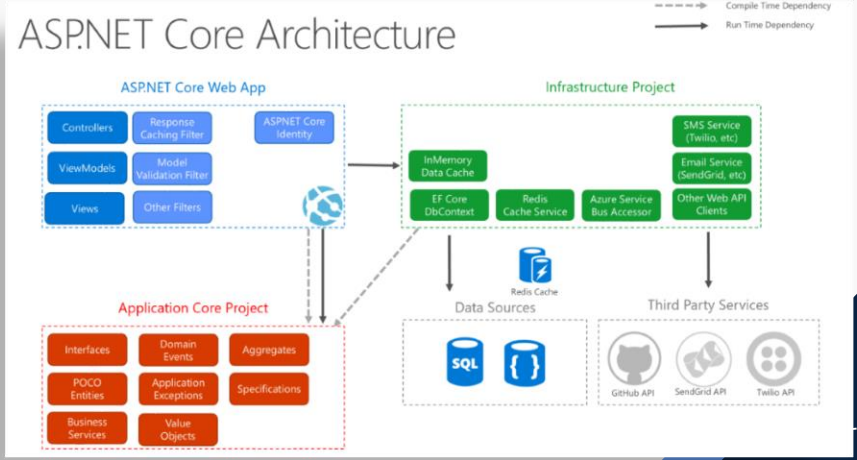
.NET FRAMEWORK

- **Platform Support:** The **.NET Framework** is restricted to **Windows**.
- **Architecture and Performance:** The **.NET Framework** uses a heavier, more **monolithic architecture** for stable Windows apps



.NET CORE

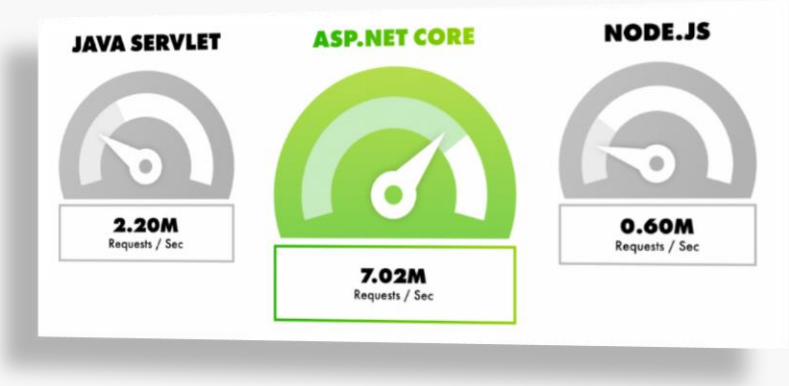
- **Platform Support:** The **.NET Core** is **cross-platform**, running on Windows, macOS, and Linux.
- **Architecture and Performance:** The **.NET Core** is **modular and lightweight**, designed for high performance in modern, cloud-based applications.



PERFORMANCE & BENCHMARK

The high performance of .net is a result of continuous optimization across the entire stack:

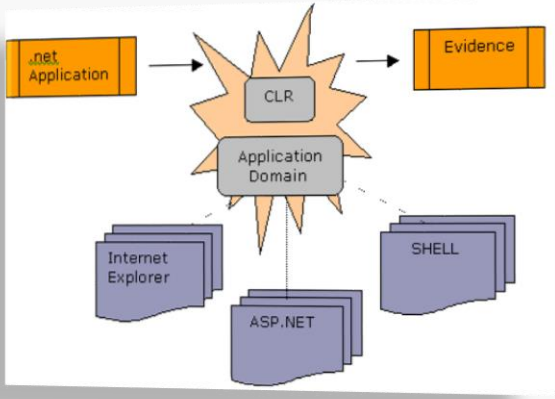
- ✓ Dynamic profile – guided optimization
- ✓ Native Ahead of time(AOT) compilation
- ✓ .Net among top 3 fastest backend technologies
- ✓ Kestrel server performance



.NET SECURITY

.Net security = protecting your app + protecting your data + protecting your users + protecting your machine.

- The .NET security framework architecture consists of the following five core elements:
  - ✓ Evidence Based Security
  - ✓ Code Access Security
  - ✓ Verification
  - ✓ Role Based Security
  - ✓ Cryptography



SUMMARY

Versatility: Allows developers to build virtually any type of application (web, mobile, desktop, cloud, or gaming).

Modernity: Utilizes modern tools and languages like C#.

Reliability: Guarantees continuous upgrades, strong performance, and cross-platform support.

Efficiency: Provides a complete ecosystem to create fast, scalable, and high-quality applications.

THANK YOU